# Übungsblatt 2: Suchen und Zählen

**[Modulname] ([Modulkürzel])**

**Musterlösung**

## Zielsetzung und Vorbereitung

Sie implementieren zwei lineare Suchverfahren, prüfen Normal- und Randfälle und begründen Korrektheit sowie Terminierung. Das Blatt ist ein didaktisches Beispiel für Informatik; geplant sind 20–30 Minuten Einzelarbeit einschließlich Tests und kurzer Auswertung, keine verbindliche Modulzeitvorgabe.

Vorausgesetzt werden Funktionen, Listen, Vergleiche und Schleifen in Python. Sie benötigen Python 3.12 und einen Texteditor; zusätzliche Pakete oder Hardware sind nicht erforderlich. Die Funktionen verarbeiten eine endliche Liste ganzer Zahlen `values` und eine ganze Zahl `target`, verändern die Liste nicht und geben eine ganze Zahl zurück. Ungültige Eingabetypen sind nicht Teil der Aufgabe. Python verwendet für ganze Zahlen keine feste Wortbreite.

Startcode herunterladen. Entpacken Sie das Archiv und öffnen Sie ein Terminal im entpackten Verzeichnis `lab_02_suchen`. Die enthaltene `README.md` beschreibt die Schritte ebenfalls.

```
python3 --version
cd starter
python3 -c "import search"
python3 ../tests/check.py
```

Die Versionsausgabe muss mit `Python 3.12.` beginnen. Verwenden Sie auf Ihrem System gegebenenfalls `python` oder `py -3.12` anstelle von `python3`, jeweils mit überprüfter Version. Der Import muss gelingen. Die Tests melden zunächst absichtlich Exitcode 1 und vier offene Testgruppen, da die beiden Funktionen noch `NotImplementedError` auslösen:

```
{
  "tests": 4,
  "failures": [
    "test_first", "test_notfound", "test_count", "test_empty"
  ],
  "errors": []
}
```

`errors` muss leer sein; Syntax- und Importfehler sind keine vorgesehenen Aufgabenlücken. Ändern Sie ausschließlich `starter/search.py`, erhalten Sie die Funktionssignaturen und lassen Sie die Tests unverändert. Implementieren Sie die Aufgaben mit `for`- oder `while`-Schleifen; `enumerate` ist erlaubt. Verwenden Sie keine fertigen Such- und Zählfunktionen wie `list.index`, `list.count` oder `sum`.

## Aufgabe 1: Ersten Treffer finden

Ergänzen Sie `first_index(values: list[int], target: int) -> int` beim Kommentar `TODO Aufgabe 1`. Die Funktion soll den nullbasierten Index des ersten Treffers liefern. Ist der Wert nicht enthalten, muss sie `-1` liefern.

| Aufruf | Erwartetes Ergebnis |
|---|---|
| `first_index([4, 7, 7, 2], 7)` | `1` |
| `first_index([7, 2, 7], 7)` | `0` |
| `first_index([4, 2], 7)` | `-1` |
| `first_index([], 7)` | `-1` |

Prüfen Sie die Beispiele und erklären Sie kurz, warum Ihre Implementierung bei mehreren Treffern den ersten liefert und bei jeder zulässigen Liste terminiert. Planen Sie hierfür etwa 10 Minuten ein.

### Musterlösung Aufgabe 1

```python
def first_index(values: list[int], target: int) -> int:
    """Ersten nullbasierten Index von target liefern, sonst -1."""
    for index, value in enumerate(values):
        if value == target:
            return index
    return -1
```

Vor jedem Schleifendurchlauf sind alle Werte vor `index` geprüft und von `target` verschieden. Ein Treffer an `index` ist deshalb der erste Treffer. Nach vollständigem Durchlaufen wurde kein Treffer gefunden, also ist `-1` korrekt. Für eine leere Liste wird die Schleife nicht betreten und ebenfalls `-1` zurückgegeben. Die Schleife besucht jeden Listeneintrag höchstens einmal; bei endlicher Liste terminiert sie. Die Liste bleibt unverändert.

Im Beispiel `[4, 7, 7, 2]` mit Zielwert `7` wird zuerst `4`, dann die erste `7` geprüft; die Rückgabe ist `1`. Eine Rückgabe von `-1` innerhalb der Schleife würde die Suche nach dem ersten unpassenden Wert zu früh beenden. Eine Rückgabe des Werts `7` statt seines Index würde den Vertrag verletzen.

## Aufgabe 2: Alle Treffer zählen

Ergänzen Sie `count_matches(values: list[int], target: int) -> int` beim Kommentar `TODO Aufgabe 2`. Die Funktion soll die Anzahl aller Vorkommen liefern, bei fehlendem Zielwert oder leerer Liste also `0`.

| Aufruf | Erwartetes Ergebnis |
|---|---|
| `count_matches([4, 7, 7, 2], 7)` | `2` |
| `count_matches([7, 7, 7], 7)` | `3` |
| `count_matches([4, 2], 7)` | `0` |
| `count_matches([], 7)` | `0` |

Prüfen Sie die Beispiele und begründen Sie, dass jedes Vorkommen genau einmal gezählt wird. Wie viele Gleichheitsvergleiche benötigen beide Funktionen im schlechtesten Fall bei einer Liste

der Länge $n$? Im verwendeten Kostenmodell zählt jeder Ganzzahlvergleich als eine Operation; die Bitlänge der Zahlen wird dabei ausgeblendet. Es geht um eine algorithmische Schranke, nicht um eine gemessene Laufzeit. Planen Sie etwa 10 Minuten ein.

**Musterlösung Aufgabe 2**

```python
def count_matches(values: list[int], target: int) -> int:
    """Anzahl aller Vorkommen von target liefern, sonst 0."""
    count = 0
    for value in values:
        if value == target:
            count += 1
    return count
```

Vor jedem Durchlauf ist `count` die Anzahl der Treffer im bereits untersuchten Präfix. Ein passender Wert erhöht den Zähler genau um eins, jeder andere Wert lässt ihn unverändert. Nach dem letzten Durchlauf enthält `count` somit die Anzahl aller Treffer. Die leere Liste hat keine Treffer: Der Zähler bleibt `0`. Die endliche Liste wird vollständig und unverändert durchlaufen; die Anzahl noch zu prüfender Elemente nimmt in jedem Durchlauf ab.

Im Beispiel `[4, 7, 7, 2]` mit Zielwert `7` nimmt der Zähler nach den einzelnen Werten die Zustände `0`, `1`, `2`, `2` an. Eine Rückgabe innerhalb der Schleife würde weitere Treffer übergehen; ein stets zurückgegebener Wert `1` wäre nur eine Aussage über das Vorhandensein eines Treffers.

`first_index` benötigt im schlechtesten Fall $n$ Vergleiche, etwa bei einem fehlenden Zielwert; bei einem ersten Treffer am Anfang genügt ein Vergleich. `count_matches` benötigt für jede Liste genau $n$ Vergleiche. Im angegebenen Kostenmodell haben beide Verfahren im schlechtesten Fall eine Zeitkomplexität von $\Theta(n)$ und zusätzlichen Speicherbedarf $\Theta(1)$, gemessen in Variablen bzw. Maschinenwörtern mit ausreichend großen Indizes und Zählern. Bei Python-Zahlen beliebiger Größe ist dies kein bitgenaues Speicher- oder Laufzeitmodell.

## Abschlussprüfung und Arbeitsergebnis

Führen Sie aus `starter/` erneut Import und Tests aus:

```
python3 -c "import search"
python3 ../tests/check.py
```

Beide Befehle sollen nun mit Exitcode 0 enden; der Test soll melden:

```
{"tests": 4, "failures": [], "errors": []}
```

Die vier Testgruppen prüfen erste Treffer, fehlende Werte, Duplikate und leere Listen. Unter `failures` erscheinen nicht erfüllte Gruppen; unerwartete Ausnahmen erscheinen unter `errors`. Bestandenes Testen ausgewählter Eingaben ersetzt keine allgemeine Korrektheitsbegründung. Halten Sie die bearbeitete `starter/search.py` sowie Ihre kurzen Begründungen gemäß den Abgabevorgaben der Lehrveranstaltung bereit. Für Test und Abschluss sind etwa 5 Minuten vorgesehen.

### Prüfung der Referenzlösung

Die vollständige ausführbare Referenz liegt in `solution/search.py` im Projektordner `code/labs/lab_02_suchen/`; sie gehört nicht zum Startcode-Archiv. Wechseln Sie im Repository in diesen Projektordner und anschließend nach `solution/`. Dort gelten dieselben Import- und Testbefehle. Die Referenz muss alle vier öffentlichen Testgruppen bestehen. Für die fachliche Prüfung wurden unter Python 3.12.14 zusätzlich alle Listen der Längen 0 bis 5 über den Werten $-1$, 0 und 1 sowie Zielwerte von $-2$ bis 2 gegen unabhängige Referenzausdrücke verglichen. Die endliche Prüfung ergänzt die obigen Begründungen und ersetzt sie nicht.